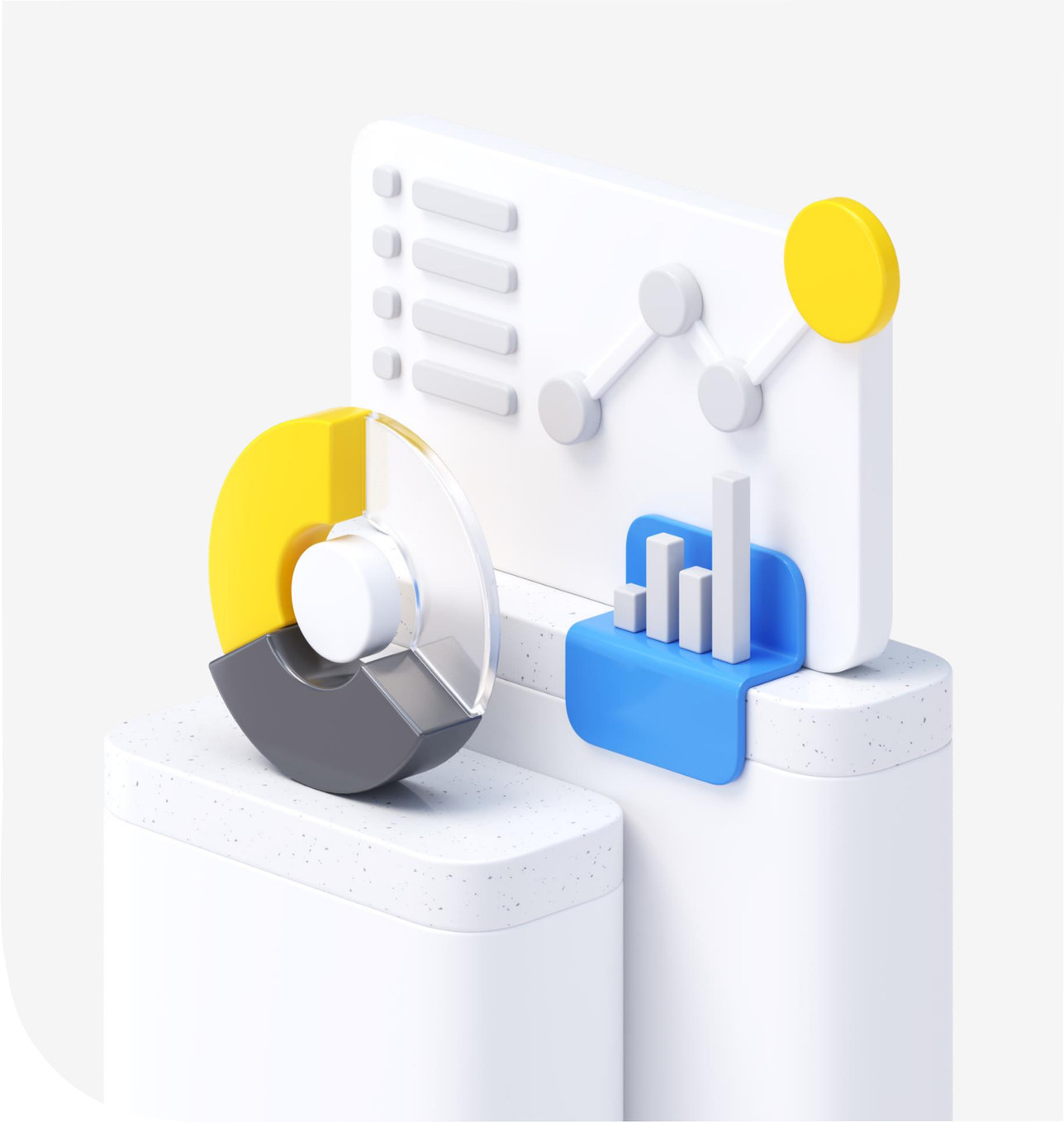
Лекция 11

Паттерны проектирования, Архитектура, DI



Темы

Паттерны проектирования

- Виды
- Области применения
- Примеры

Архитектура

- Что это такое и зачем?
- SOA
- MVC, MVP, MVVM, VIPER
- Coordinator

Инъекция зависимостей

- Что за зверь?
- Области применения
- Примеры

Паттерны проектирования

Design Patterns: Elements of Reusable Object-Oriented Software

Приёмы объектно-ориентированного проектирования.

Па́ттерны проектирования. 1994г.

Gang of Four:

- Erich Gamma
- Richard Helm
- Ralph Johnson
- John Vlissides

Паттерны проектирования

Основные виды шаблонов

Структурные

С другой стороны рамки и место обучения кадров требуют от нас анализа систем массового участия.

Порождающие

Контролируют процесс создания объектов

Поведенческие

С другой стороны рамки и место обучения кадров требуют от нас анализа систем массового участия.

Порождающие паттерны

Имеют дело с созданием объектов

- Строитель (Builder)
- Абстрактная фабрика
- Singleton
- Ленивая инициализация (Lazy initialization)

- Определяем небольшой набор поведений, с помощью композиции которых можно получать любое число более сложных.
- Инкапсуляция знаний о конкретных классах, необходимых для создания того или иного объекта.

Порождающие паттерны

Абстрактная фабрика

Предоставляет интерфейс для создания семейств взаимосвязанных или взаимозависимых объектов, не специфицируя их конкретных классов

Порождающие паттерны

Абстрактная фабрика

```
protocol AbstractProduct {
    func usefulFunction()
}

protocol AbstractFactory {
    func createProduct() -> AbstractProduct
}

class ProductA: AbstractProduct {
    func usefulFunction() {
        print("I'm product A")
    }
}

class ProductB: AbstractProduct {
    func usefulFunction() {
        print("I'm product B")
    }
}
```

```
class FactoryA: AbstractFactory {
    func createProduct() -> AbstractProduct {
        ProductA()
    }
}

class FactoryB: AbstractFactory {
    func createProduct() -> AbstractProduct {
        ProductB()
    }
}
```

```
var factory: AbstractFactory = FactoryA()
factory.createProduct().usefulFunction() // I'm product A

factory = FactoryB()
factory.createProduct().usefulFunction() // I'm product B
```

Порождающие паттерны

Singleton

Шаблон проектирования, гарантирующий, что в однопоточном приложении будет единственный экземпляр некоторого класса, и предоставляющий глобальную точку доступа к этому экземпляру.

```
class Singleton {
    static var shared: Singleton = {
        let instance = Singleton()
        return instance
    }()

    private init() {}

    func someBusinessLogic() -> String {
        "Hello, world!"
    }
}

let a = Singleton.shared
let b = Singleton.shared

print(a === b) // true
```

Порождающие паттерны

Singleton

Шаблон проектирования, гарантирующий, что в однопоточном приложении будет единственный экземпляр некоторого класса, и предоставляющий глобальную точку доступа к этому экземпляру.

Минусы:

- Все время в памяти
- Требует отдельного подхода к тестированию
- Усложняется работа с многопоточностью

```
class Singleton {
    static var shared: Singleton = {
        let instance = Singleton()
        return instance
    }()

    private init() {}

    func someBusinessLogic() -> String {
        "Hello, world!"
    }
}

let a = Singleton.shared
let b = Singleton.shared

print(a === b) // true
```

Структурные паттерны

Отвечают за построение удобных в поддержке иерархий классов

- Адаптер
- Декоратор
- Фасад
- Мост (Bridge)

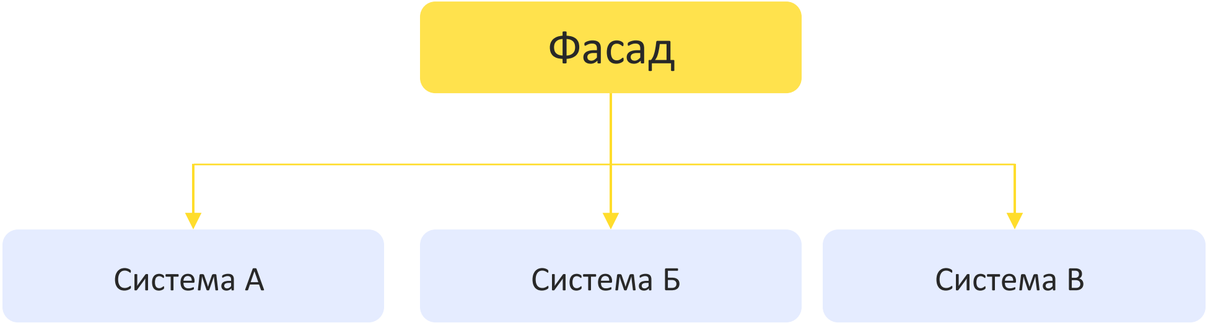
Примеры применения:

- Организация совместной работы нескольких независимо разработанных библиотек
- Образование из объектов и классов более крупных структур

Структурные паттерны

Фасад

Позволяет скрыть сложность системы путём сведения всех возможных внешних вызовов к одному объекту, делегирующему их соответствующим объектам системы.



Структурные паттерны

Адаптер

Позволяет объектам с несовместимыми интерфейсами работать вместе.

```
class NetworkService {
    func makeRequest() {•••}
}

class MyAwesomeService {
    func makeAwesomeRequest() {•••}
}

class NetworkAdapter: NetworkService {
    let awesomeService: MyAwesomeService

    override func makeRequest() {
        awesomeService.makeAwesomeRequest()
    }
}
```

Поведенческие паттерны

Определяют алгоритмы и способы реализации взаимодействия объектов и классов

- Цепочка ответственности (Chain of Responsibility)
- Наблюдатель (Observer)
- Стратегия (Strategy)
- Хранитель (Memento)

Поведенческие паттерны

Наблюдатель

Реализует у класса механизм, который позволяет объекту этого класса получать оповещения об изменении состояния других объектов и тем самым наблюдать за ними

```
class Incrementer {
    private var value = 0

    func increment() {
        value += 1
        NotificationCenter.default.post(
            name: Notification.Name("DidIncrement"),
            object: nil,
            userInfo: ["newValue": value]
        )
    }
}

let incrementer = Incrementer()

NotificationCenter.default.addObserver(
    forName: Notification.Name("DidIncrement"),
    object: nil,
    queue: .main
) { notification in
    guard let newValue = notification.userInfo?["newValue"] else {
        return
    }
    print(newValue)
}

incrementer.increment()
```

Структурные паттерны

Стратегия

Предназначен для определения семейства алгоритмов, их инкапсуляции и обеспечения их взаимозаменяемости

```
protocol CompressionStrategy {
    func compess(data: Data) -> Data
}

class RarStrategy: CompressionStrategy {•••}
class ZipStrategy: CompressionStrategy {•••}

class Archiver {
    let strategy: CompressionStrategy
    init(strategy: CompressionStrategy) {•••}
    func archive(data: Data) -> Data {
        let compressed = strategy.compess(data: data)
        // Делаем всякие важные штуки
        return compressed
    }
}
```

Архитектура

Что такое архитектура?

Архитектура программного обеспечения — совокупность важнейших решений об организации программной системы.

Википедия

Когда нужна архитектура?

- Тяжело добавлять фичи

  Любое изменение требует много трудозатрат

- Сложно поддерживать старые

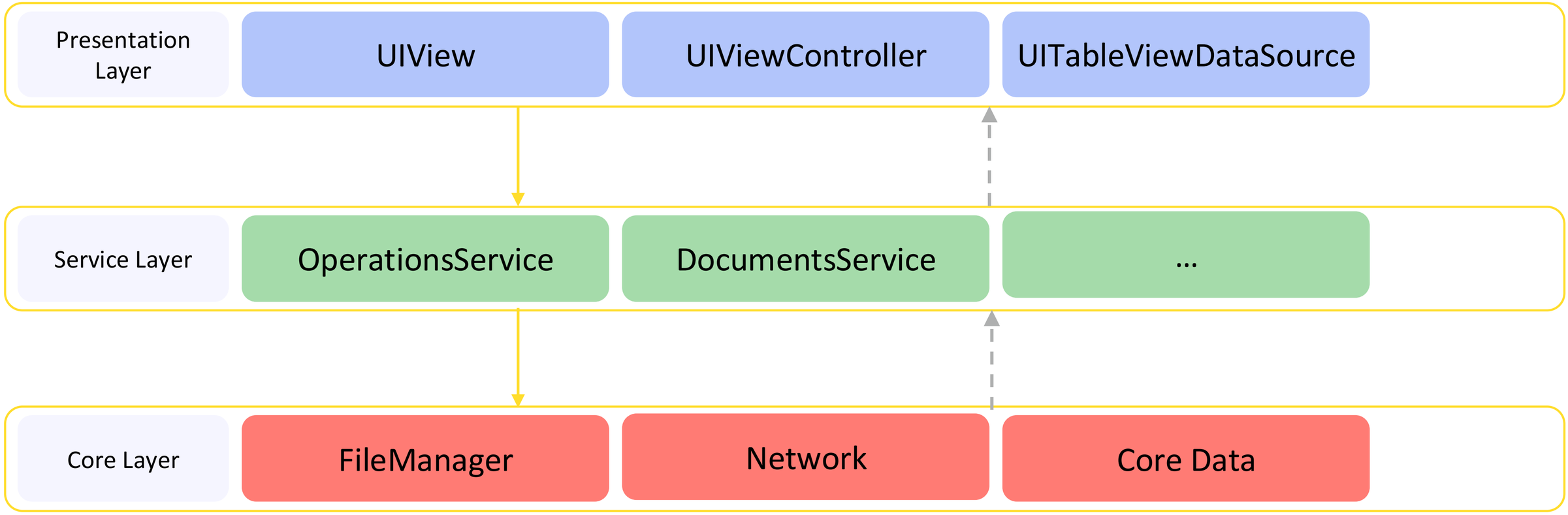
  Изменения влияют на функционал влиять на который не должны

- Тяжело или невозможно автоматизировать тестирование

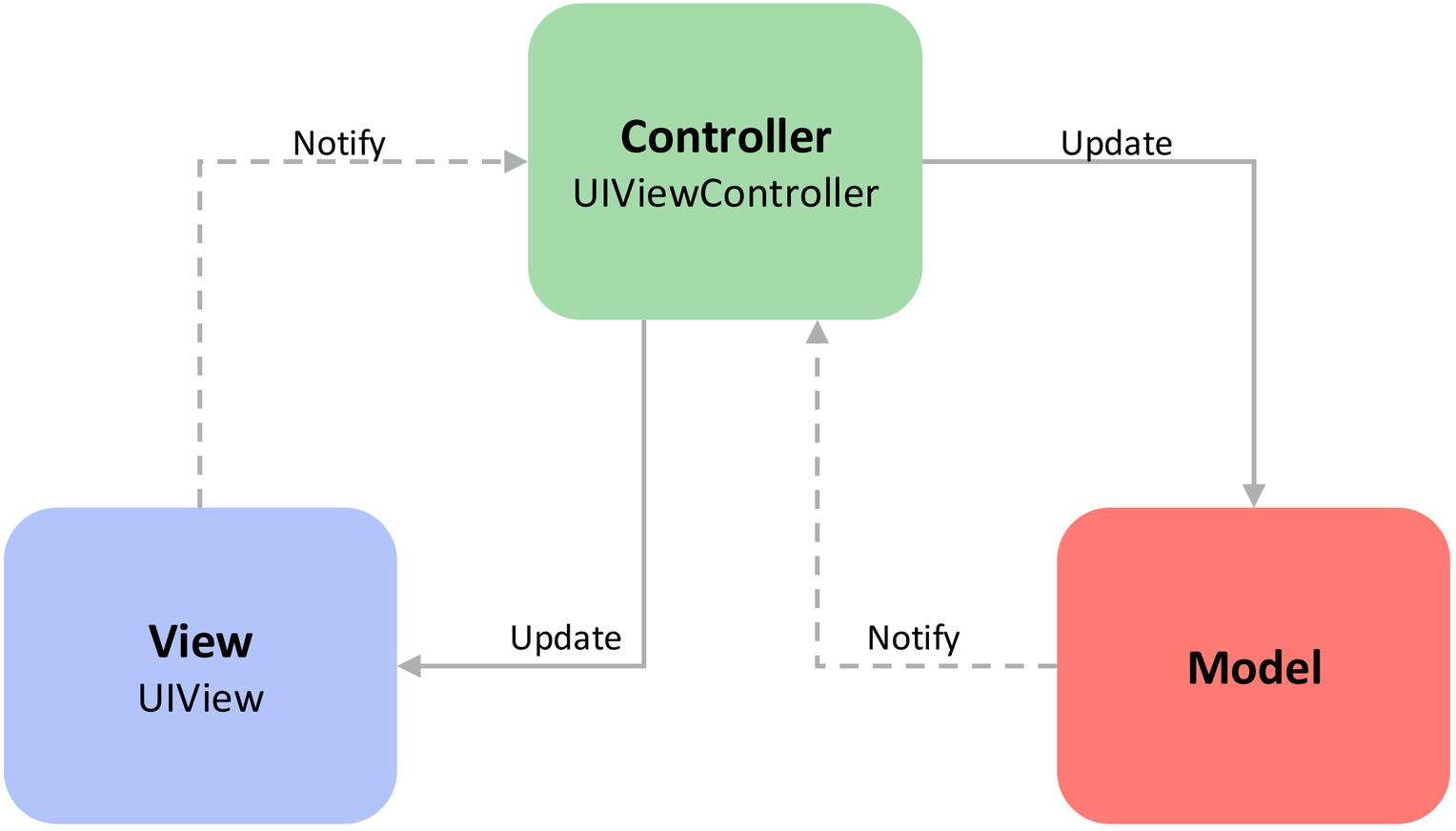
Пишите код так, будто поддерживать его будет
склонный к насилию психопат, который знает где вы живете

John F. Woods

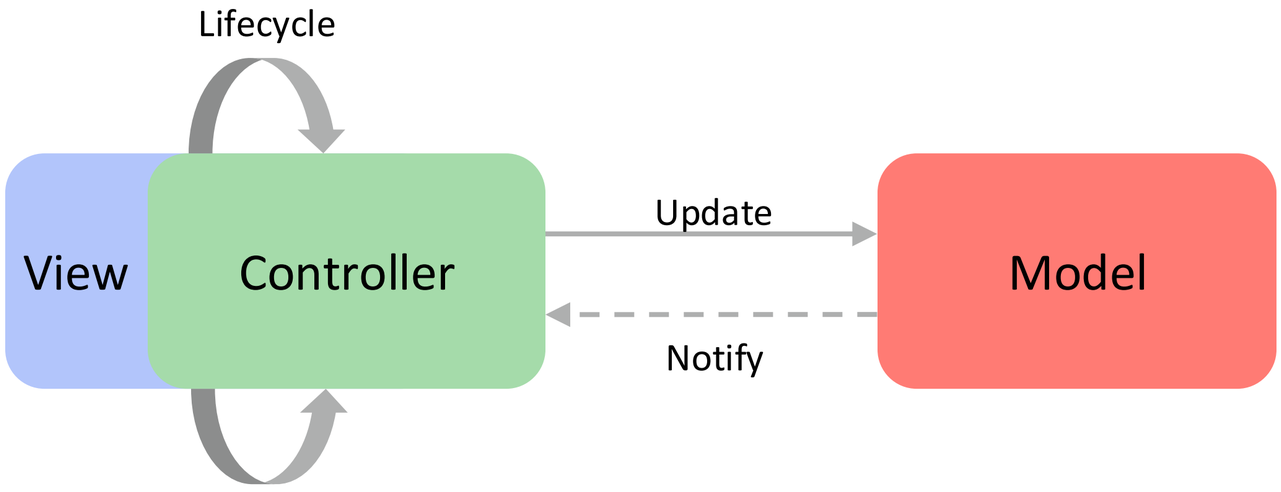
Сервис-ориентированная архитектура



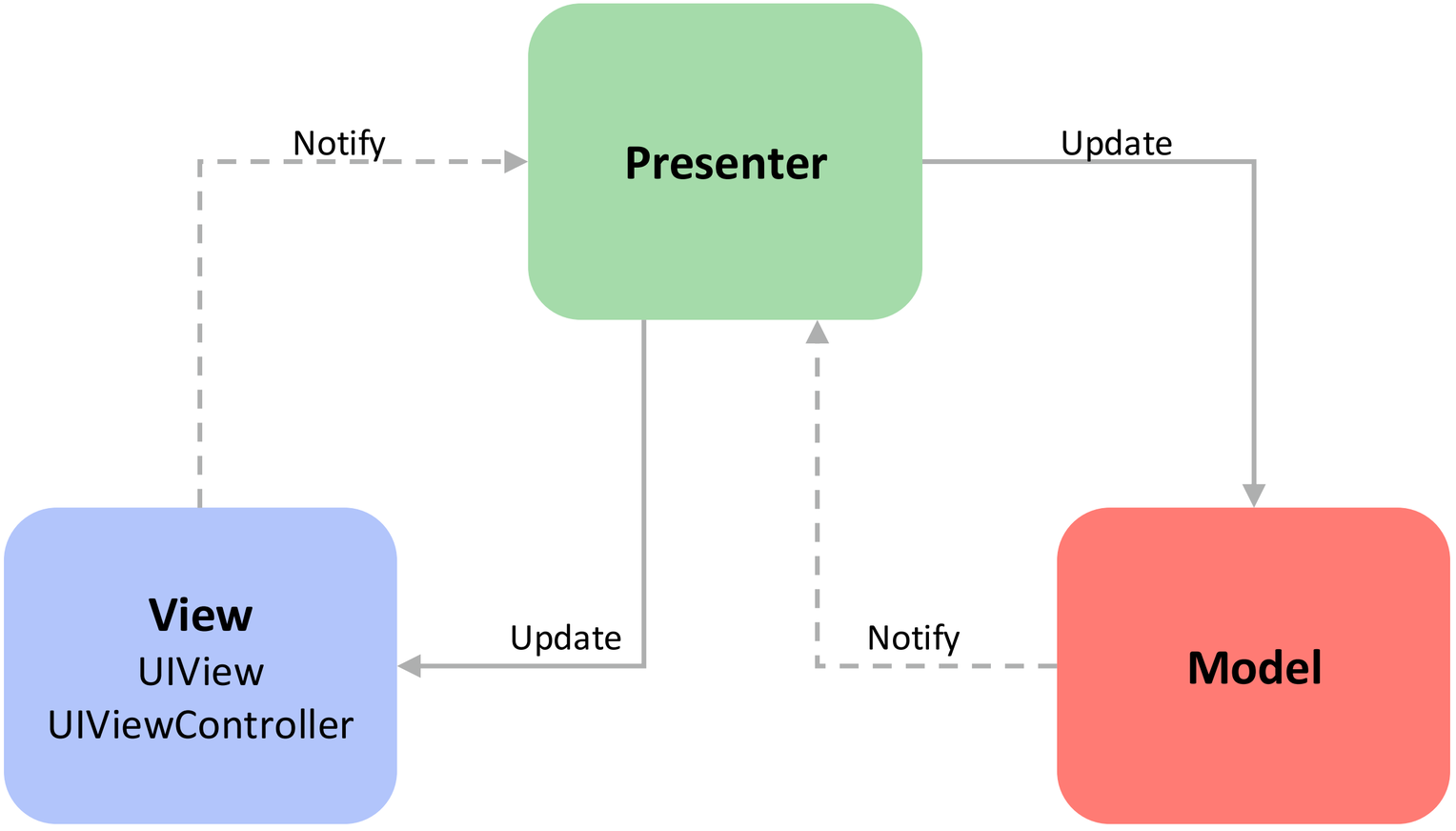
Model View Controller



Massive View Controller



Model View Presenter

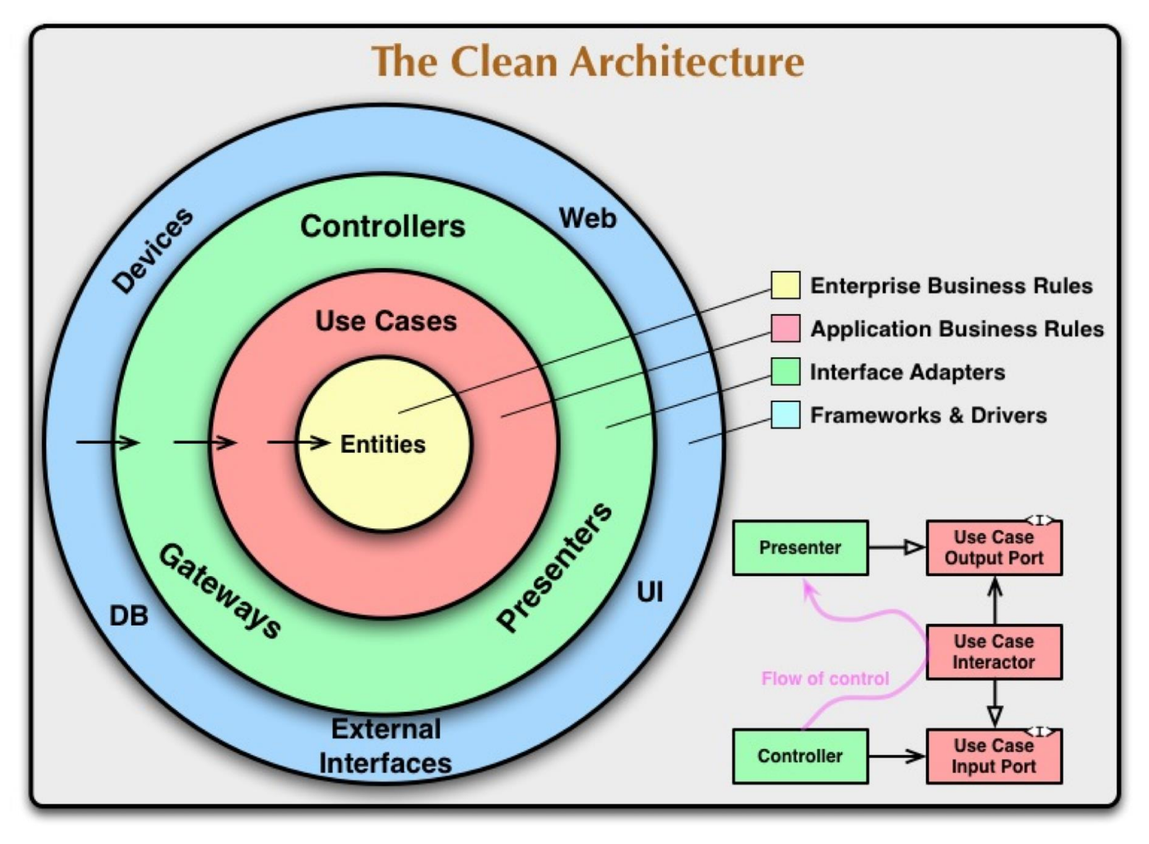


Clean Architecture

Бизнес-логика

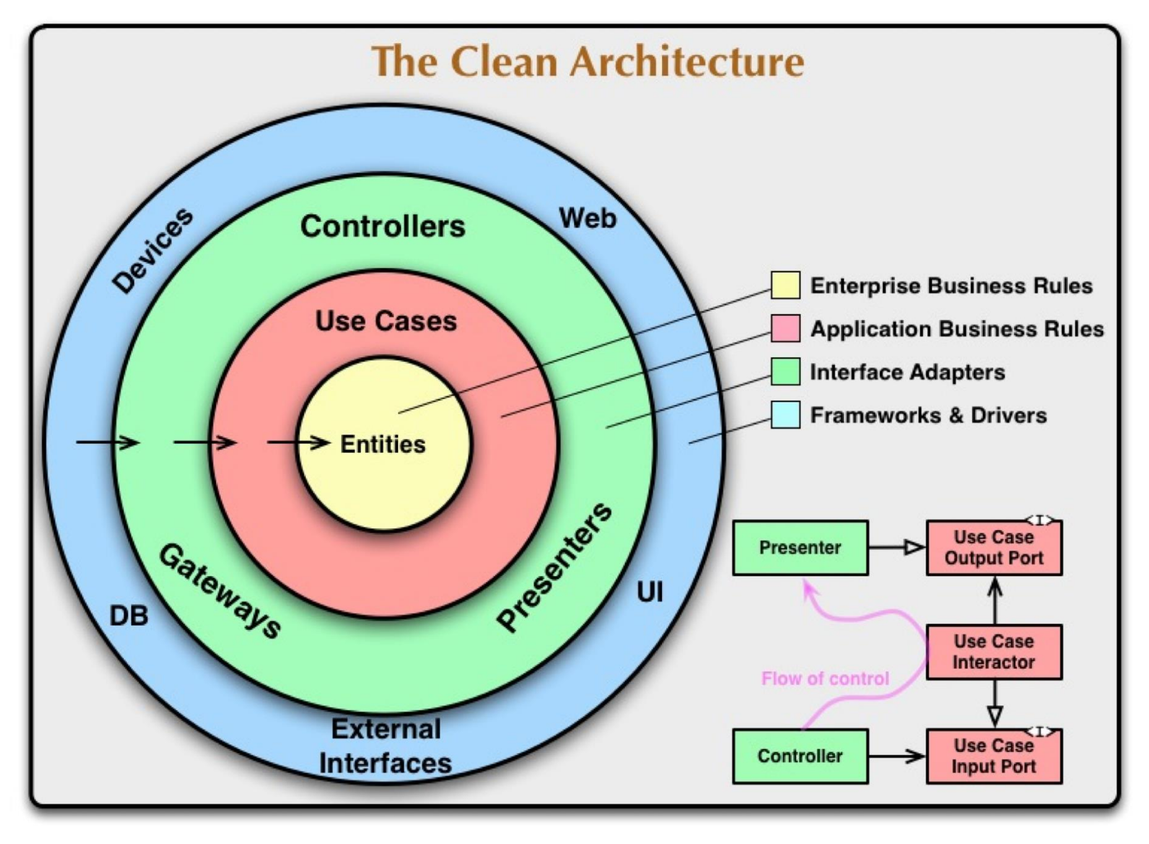
- Логика предметной области (Enterprise Business Logic) – описывает бизнес-процессы
- Логика приложения (Application Logic) – описывает процессы и потоки данных внутри приложения

Clean Architecture



Ничто во внутреннем круге не может знать или как-либо ссылаться на что-либо во внешнем круге

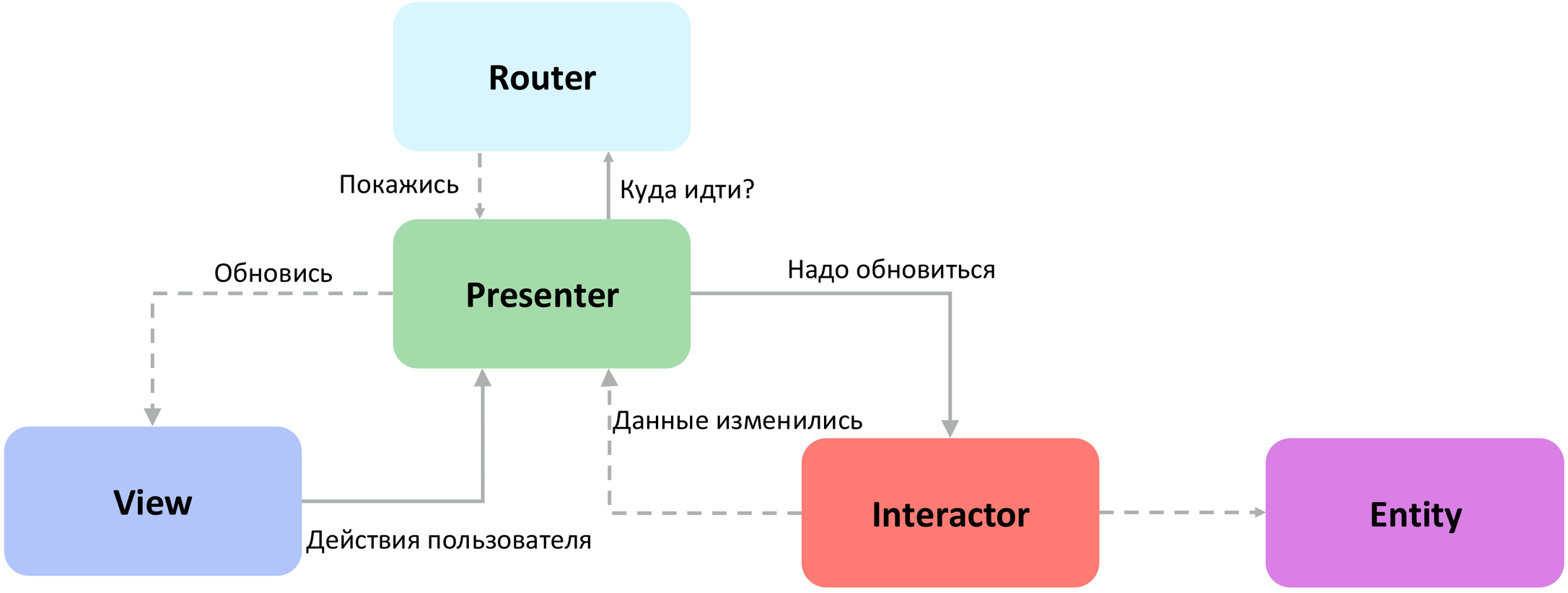
Clean Architecture



- Сущности (Entities) - Инкапсулируют логику предметной области. Именно тут заложены самые высокоуровневые правила бизнеса.
- Use Cases – Содержат логику приложения. Управляют потоком данных к и от Entities и дают возможность энтерпрайз логике работать
- Interface Adapters – Набор адаптеров которые конвертируют данные из формата удобного для Use Case в формат пригодный для баз данных, сети или UI.
- Frameworks & Drivers – Это соответственно слой содержащий конкретные реализации баз данных, сети и прочего.

VIPER

View Interactor Presenter Entity Router



VIPER

Плюсы

- Высокая степень тестируемости
- Разделенность компонент
- Соответствие принципам проектирования

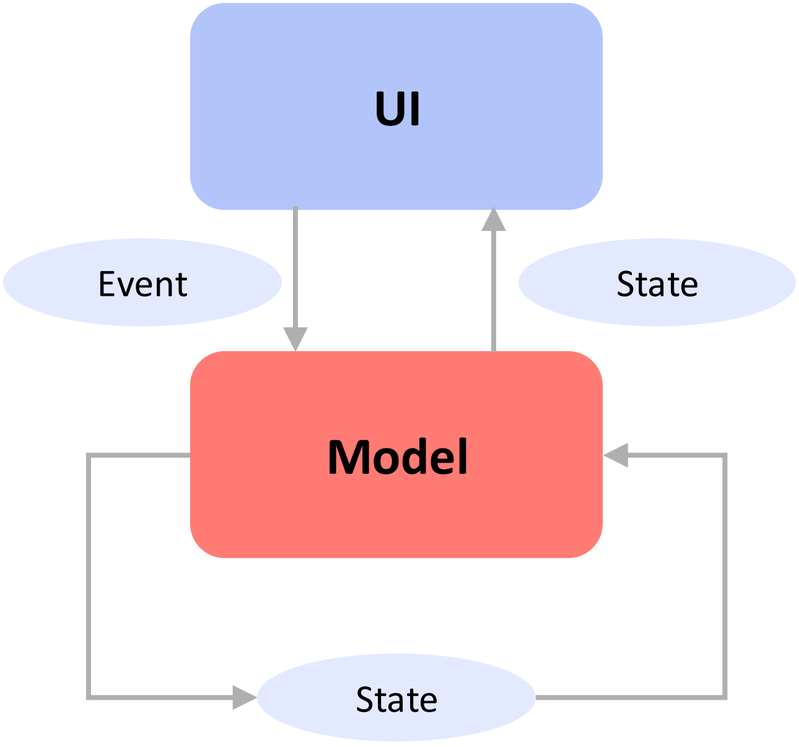
VIPER

Минусы

- Очень много кода
- Плохо читается
- Частая зависимость от сторонних утилит

UDF

Unidirectional Data Flow



Итоги

- Нет одной правильной архитектуры
- Не усложняйте себе жизнь
- Эволюция – это нормально

Инъекция зависимостей

Dependency Injection

Dependency Injection

Инъекция зависимостей - паттерн настройки объекта,
при котором зависимости объекта задаются извне,
а не создаются самим объектом.

- Interface injection
- Property injection
- Method injection
- Constructor injection

Инъекция через интерфейс

```
final class Presenter {

    private lazy var userService = getUserService()

    func getData() -> Data {
        return userService.getData()
    }
}
protocol PresenterResolving {
    func getUserService() -> UserServiceProtocol
}
extension Presenter: PresenterResolving {
    func getUserService() -> UserServiceProtocol { return UserService() }
}
```

- Interface injection
- Property Injection
- Method injection
- Constructor injection

Инъекция через публичное свойство

```
final class Presenter {
    var userService: UserServiceProtocol?
}
final class UserService:  UserServiceProtocol {•••}

let presenter = Presenter()
presenter.userService =  UserService(...)
```

- Interface injection
- Property injection
- **Method injection**
- Constructor injection

Инъекция через публичный метод

```
final class Presenter {
    private var interactor: InteractorInput?

    func set(_ interactor: InteractorInput) {
        self.interactor = interactor
    }
}
final class Interactor: InteractorInput {•••}

let interactor = Interactor()
presenter.set(interactor)
```

- Interface injection
- Property injection
- Method injection
- Constructor injection

Инъекция через конструктор

```
final class OperationService {

    private let networkService: NetworkServiceProtocol
    private let accountNumber: String

    init(network: NetworkServiceProtocol, accountNumber: String) {
        self.userService = userService
        self.modelId = modelId
    }

    func getOperations(_ completion: @escaping ([Operation] -> Void)) {•••}
}
final class NetworkService:  NetworkServiceProtocol {•••}

let network: NetworkServiceProtocol = NetworkService()
let service = OperationService(network: network, accountNumber: "1234567890")
```

Виды DI фреймворков

Runtime

Плюсы

- Гибкость
- Простота конфигурации
- Использование в условиях неопределенности

Минусы

- Производительность
- Сложность отладки
- Усложнение структуры приложения

Виды DI фреймворков

Compile-Time

Плюсы

- Безопасность типов
- Производительность
- Предсказуемость

Минусы

- Гибкость
- Сложность конфигурации
- Кодозависимость

Swinject

```
let container = Container()

container.register(FooRepository.self) { _ in

 FooRepositoryImpl()

}

container.register(FooUseCase.self) { r in

 SwinjectUseCase(fooRepository: r.resolve(FooRepository.self)!)

}


let useCase = container.resolve(FooUseCase.self)
```

Needle

```
class NeedleComponent: BootstrapComponent {
  var fooRepository: FooRepository {
    shared { FooRepositoryImpl() }
  }
  var fooUseCase: FooUseCase {
    shared { NeedleUseCase(parent: self) }
  }
}
final class NeedleUseCase: Component<NeedleDependency>, FooUseCase {
  var fooRepository: FooRepository { dependency.fooRepository }
  func fetch() -> String {
    return fooRepository.fetchData()
  }
}
```

Плюсы DI

01

Делает зависимости объекта явными, четко прописанными в интерфейсе объекта — это позволяет лучше контролировать его сложность.

02

Зависимости – внешние. Объект не ответственен за их создание, соответственно содержит только код, относящийся к его бизнес логике.

03

Зависимости – гибкие. Можно подменить объект другим, закрыв его протоколом. Классы начинают зависеть от абстракций, а не от конкретной реализации. Повышается тестируемость.

04

Упрощается переиспользование объектов

05

Уменьшается связанность кода.

Минусы DI

01

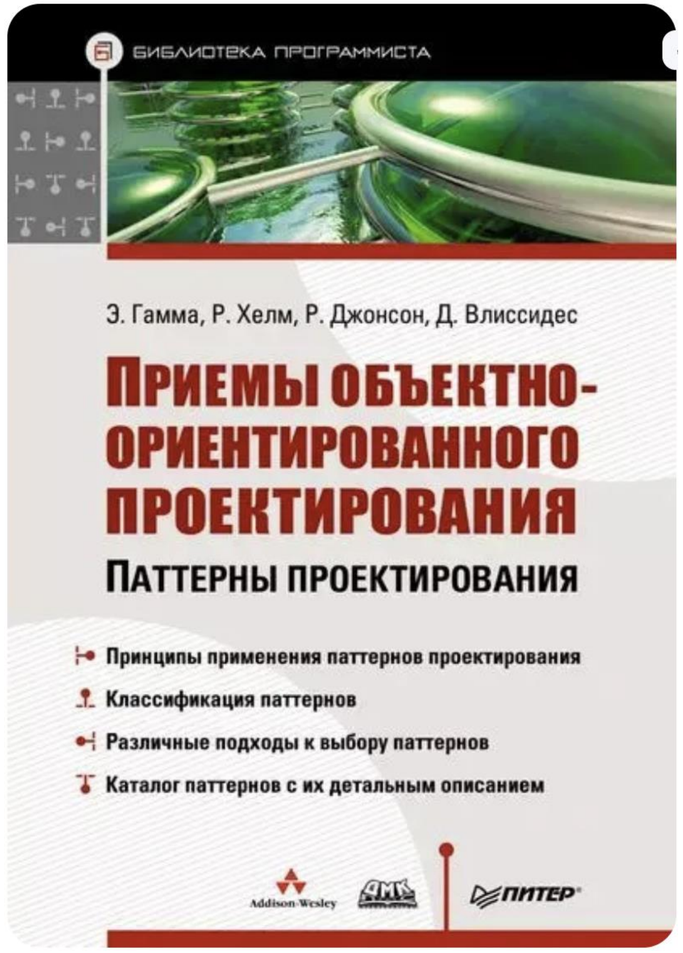
Становится больше объектов,
Добавляются сущности с
порождающей логикой, протоколы

02

Увеличивается время на
разработку

Список литературы

- Приёмы объектно-ориентированного проектирования. Па́ттерны проектирования.
- Введение в DI (Habr)
- Чистая архитектура
- Swinject
- Needle



Т БАНК

Спасибо!